.NET Core vs .NET Framework Code Execution Process

Back to .NET Core Tutorials for Beginners

.NET Core vs .NET Framework Code Execution Process

In this article, I will discuss the **.NET Core vs .NET Framework Code Execution Process**. So, we will understand the .NET Framework Code Execution Process in comparing to the .NET Core .NET Framework Application Execution Process and let's understand what changes have been made in the .NET Core.

.NET Framework Code Execution Process

Let us first discuss the .NET Framework Code Execution Process. The code execution process of .NET Framework Application involves the following 4 steps.

- **Choosing a Compiler**
- **Compiling Both Source Code to MSIL**
- **Compiling MSIL to Native Code**
- **Running Native Code**

For a better understanding, check these out in the following diagram.

.NET Framework Code Execution Process

Choosing a Compiler:

The .NET Framework allows us to develop applications using multiple programming languages, and each programming language has its own compiler. Depending on the language we choose, the respective language compiler will compile the source code into intermediate language.

For example, if you're building a Console Application or Desktop Application, the IL code is saved in an .EXE file.

If you're building a Class Library Application, the IL code is saved in a .DLL file.

In the .NET Framework, you can check the MSIL Code and Metadata using ILDasm utility. For example, if you want to see the MSIL Code and Metadata of the Class Library Application, then open the ILDasm utility and open the Assembly and accessing the MSIL Code and Metadata.

Compiling IL or MSIL to Native Code:

When we run our .NET Framework application, the .NET Artifacts (.EXE or .DLL) are loaded into Common Language Runtime (CLR). The Common Language Runtime (CLR) is responsible for managing the execution of .NET Framework Applications.

The process of converting IL or MSIL Code to native code happens at runtime just before the code is executed, allowing the JIT compiler to optimize the code based on the system's hardware, ensuring that it can be executed efficiently by the operating system.

Running Code:

After the JIT Compiler converts IL code into Native Machine Code, the operating system can execute the code directly. During execution, the CLR provides the infrastructure needed for the code, including memory and security.

- Automatic Memory Management and Garbage Collection to handle memory allocation and deallocation.
- Security Enforcement to ensure code operates within defined security boundaries.
- Allows code to be managed in different languages (like C#, VB, F#) seamlessly.

The combination of IL code and the CLR's runtime services ensures that applications built with the .NET Framework can run efficiently on various platforms and devices.

Points to Remember:

In the .NET Framework, the Base Class Library (BCL) also known as the Framework Class Library (FCL), provides a rich set of built-in classes and functionalities for tasks like data access, networking, file system, etc.

- FCL includes libraries for things like UI data access, networking, file system, etc.
- BCL is a collection of pre-compiled classes that can be used in the underlying operating system for various tasks.

The process of converting IL or MSIL Code to native code happens at runtime just before the code is executed, allowing the JIT compiler to optimize the code based on the system's hardware, ensuring that it can be executed efficiently by the operating system.

Running Code:

After the JIT Compiler converts IL code into Native Machine Code, the operating system can execute the code directly. During execution, the CLR provides the infrastructure needed for the code, including memory and security.

- Automatic Memory Management and Garbage Collection to handle memory allocation and deallocation.
- Security Enforcement to ensure code operates within defined security boundaries.
- Allows code to be managed in different languages (like C#, VB, F#) seamlessly.

The combination of IL code and the CLR's runtime services ensures that applications built with the .NET Framework can run efficiently on various platforms and devices.

.NET Core Code Execution Process

Now, let us see the .NET Core Code execution process. The code execution process of .NET Core Application involves the following 4 steps.

- **Choosing a Compiler**
- **Compiling Both Source Code to MSIL**
- **Compiling MSIL to Native Code**
- **Running Native Code**

For a better understanding, check these out in the following diagram.

.NET Core Code Execution Process

Choosing a Compiler:

Similar to the .NET Framework, when we develop applications using multiple programming languages like C#, VB and Visual Basic (VB), Each programming language has its own compiler.

- For C# file compiler used is part of Roslyn.
- For VB file compiler is also part of Roslyn.
- For F# file compiler used is part of F# language. (.fsc.exe).

Depending on your chosen language, the appropriate language compiler will connect your source code into intermediate language (i.e. C# Intermediate Language (CIL), which can be compared to the .NET IL).

Compiling Source Code to IL or MSIL Code:

Once you write the source code in your chosen language (VB, F#, or C#), the language compiler (e.g. C# Compiler, VB Compiler, F# Compiler) converts the source code into intermediate language (i.e. C# IL, VBIL, which is platform-independent code) and saves it in a file with the extension .exe or .dll. The .exe file is for a console application, and the .dll file is for a library.

Compiling IL or MSIL to Native Code:

- If you build a console or desktop application, the IL code is saved in an .EXE file.
- If the library or web application, it is saved in a .DLL file.

In .NET Core, the IL code is also called Managed Code because it is managed by the Core CLR (Common Language Runtime). The Core CLR provides services like memory management, garbage collection, and security.

Compiling IL or MSIL to Native Code:

When you run your .NET Core application, the .NET Core runtime (Core CLR) loads the IL/MSIL Code from the .EXE or .DLL files. The Core CLR then converts the IL/MSIL code into machine code (native code).

The Just-in-Time (JIT) Compiler (RyuJIT) in the Core CLR converts the IL/MSIL Code into Native Code (also called Machine Code). This conversion occurs at runtime, allowing the JIT compiler to optimize the code based on the system's hardware, ensuring that it can execute efficiently on the specific environment. Hardware, and operating system.

The process of converting IL to native code ensures that the code is optimized for the best possible performance on the specific machine it is running on.

Running Code:

After the JIT Compiler converts IL code into Native Code, the operating system can execute it directly. During execution, the Core CLR provides essential services:

- Automatic Memory Management and Garbage Collection to manage memory allocation and deallocation.
- Security Enforcement to ensure code operates within defined security boundaries.

This process ensures that applications developed using .NET Core can run efficiently across different platforms, leveraging the JIT compilation and runtime services for optimal performance.

Changes in .NET Core:

The execution process remains the same. Only a few components are changed compared with the .NET Framework.

Compilers in .NET Core:

In .NET Core, the compilers have been modernized, and many parts have been made open-source.

- **Roslyn:** The Roslyn compiler is used for both C# and VB. Roslyn is a powerful and modern compiler that provides a complete as-a-service model, which means you can use it not only to compile code but also for tasks like code analysis, refactoring, and providing IntelliSense in your development environment.
- **F# Compiler:** For the F# compiler, we use fsc.exe. It is now integrated with the .NET Core ecosystem, ensuring seamless integration with .NET Core.

Note: The key difference here is that .NET Core uses the Roslyn compiler platform, which is faster, more extensible, and provides richer tooling support compared to the compilers used in the .NET Framework.

Class Libraries in .NET Core:

In .NET Core, we have a new set of libraries called CoreFX. CoreFX is a reimplementation of the Framework Class Libraries (FCL) or Base Class Libraries (BCL), aiming to be lightweight, modular, and cross-platform. The CoreFX libraries are designed to ensure you can add only the packages that your application needs, leading to smaller, more efficient applications.

The CoreFX library is portable and cross-platform, supporting a wide ecosystem on Windows, macOS, and Linux. Unlike the .NET Framework libraries, CoreFX is distributed via NuGet, allowing developers to manage dependencies more flexibly and efficiently.

CLR (Common Language Runtime) in .NET Core:

In .NET Core, we have a new runtime called CoreCLR (CoreCLR). It is a reimplementation of the CLR specifically designed for lightweight, modular, and cross-platform. It works on Windows, Linux, and macOS, enhancing the CLR's .NET Framework in Windows-only. The CoreCLR provides a similar set of services as the .NET Framework's CLR, such as garbage collection, memory management, type safety, and security.

RyuJIT is the new version of the JIT compiler used in CoreCLR. RyuJIT is designed to be faster, more efficient, and more modular than the JIT compiler in the .NET Framework. It supports advanced optimizations and is more adaptable to different hardware and operating systems, ensuring better performance and scalability in modern applications.

Now, the process operation that should come to your mind is why we have to take the implementation of all these components that are changed in .NET Core. The answer is that we want to make the .NET Framework implement .NET Core, so Type Source and Cross Platform.

In the next article, I will discuss the **Differences Between the .NET Core vs .NET Framework**. In this article, I explain the **.NET Core vs .NET Framework Code Execution Process**. I hope you enjoy the .NET Core vs .NET Framework Code Execution Process article.

Dot Net Tutorials
About the Author: Pranaya Rout
Pranaya Rout has published thousands of articles on Dot net, Java, Python, Angular that are very great features with more than 2 million views monthly. As .NET MVP, C#, .NET Core MVP, Microsoft Azure Developer, Author, Speaker, Blogger, Microsoft Azure Cloud Computing, Microservices, Design Patterns and Self-learning lover.

Previous Lesson: How to Download and Install Postman and Fiddler
Next Lesson: Differences Between .NET Core vs .NET Framework

Leave a Reply
Your email address will not be published. Required fields are marked *
Comment *
Name *
Email *
Website
Post Comment

Online Training Program
Live Instructor Led Training Program
Live Online Training
Live Beta .NET Online Training
Live Beta C# Online Training
Pro Courses
Pro Course Access
Pro Course Bundle Training
Pro Bundle
ASP.NET Core / Entityframework & Microservices
ASP.NET Core Fundamentals
ASP.NET Core Middleware Fundamentals
ASP.NET Core Entityframework Core Fundamentals
ASP.NET Core Web Application Development
ASP.NET Core Web API Development
ASP.NET Core Microservices Development
ASP.NET Core Microservices Deployment
ASP.NET Core Microservices Monitoring
ASP.NET Core Microservices Security
ASP.NET Core Microservices Scaling
ASP.NET Core Microservices Advanced Topics
ASP.NET Core Microservices Project
Advanced Microsoft Azure Cloud Services
ASP.NET Core Microservices Project Development Process
ASP.NET Core Microservices Project Deployment Process
Advanced Microsoft Azure Cloud Services Project Development
ASP.NET Core Basics
ASP.NET Core Basics
ASP.NET Core Project Structure
ASP.NET Core Startup Class
ASP.NET Core Middleware
ASP.NET Core Middleware Pipeline
ASP.NET Core Middleware Processing Pipeline
ASP.NET Core Middleware Short Circuit
Static Files Middleware in ASP.NET Core
ASP.NET Core Routing
ASP.NET Core Routing Middleware
ASP.NET Core Routing Middleware in ASP.NET Core
ASP.NET Core Command Line Interface
ASP.NET Core Command Line Interface Commands
ASP.NET Core Application Deployment
ASP.NET Core Hosting
ASP.NET Core Hosting Models
ASP.NET Core Kestrel Web Server
ASP.NET Core Kestrel Web Server Configuration
ASP.NET Core Kestrel Web Server Processing Pipeline
ASP.NET Core Kestrel Web Server Performance Tuning
ASP.NET Core Kestrel Web Server Security
ASP.NET Core Kestrel Web Server Scaling
ASP.NET Core Kestrel Web Server Advanced Topics
ASP.NET Core Kestrel Web Server Project
ASP.NET Core Kestrel Web Server Project Development Process
ASP.NET Core Kestrel Web Server Project Deployment Process
ASP.NET Core Dependency Injection
ASP.NET Core Dependency Injection Lifetime
ASP.NET Core Dependency Injection Advanced Topics
ASP.NET Core Dependency Injection Project
ASP.NET Core Dependency Injection Project Development Process
ASP.NET Core Dependency Injection Project Deployment Process
ASP.NET Core Logging
ASP.NET Core Logging Providers
ASP.NET Core Logging Advanced Topics
ASP.NET Core Logging Project
ASP.NET Core Logging Project Development Process
ASP.NET Core Logging Project Deployment Process
ASP.NET Core Configuration
ASP.NET Core Configuration Providers
ASP.NET Core Configuration Advanced Topics
ASP.NET Core Configuration Project
ASP.NET Core Configuration Project Development Process
ASP.NET Core Configuration Project Deployment Process
ASP.NET Core Error Handling
ASP.NET Core Error Handling Middleware
ASP.NET Core Error Handling Middleware Pipeline
ASP.NET Core Error Handling Middleware Advanced Topics
ASP.NET Core Error Handling Middleware Project
ASP.NET Core Error Handling Middleware Project Development Process
ASP.NET Core Error Handling Middleware Project Deployment Process
ASP.NET Core Web Application Development Process
ASP.NET Core Web Application Development Process Advanced Topics
ASP.NET Core Web Application Development Process Project
ASP.NET Core Web Application Development Process Project Development Process
ASP.NET Core Web Application Development Process Project Deployment Process
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Development Process
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Development Process
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Development Process
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Development Process
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Development Process
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Development Process
ASP.NET Core Web Application Development Process Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment Process Advanced Topics Project Deployment